

AdaptOBE

Software Design Document (SDD) & SDLC Documentation

Intelligent Outcome-Based Education Attainment & Adaptive Learning Platform

*Department of Computer Science, UBIT
(Umaer Basha Institute of Information Technology)*

*Final Year Project — Software Design Document
Status: Modules 0–4 implemented, tested and demo-verified; Modules 5–7 planned*

Generated from the live AdaptOBE codebase. All five diagrams were validated through the Mermaid rendering toolchain before inclusion.

1. Introduction

1.1 Purpose

This document describes the software design of AdaptOBE: its architecture, data model, AI subsystems, and core workflows. It is written from the actual, working implementation — every diagram and claim in this document is verified against the real codebase and, where noted, against live-tested runtime behavior, not a proposed or aspirational design.

1.2 Scope

- **CLO → PLO mapping**, using NLP sentence embeddings instead of manual, subjective judgment.
- **Direct attainment calculation**, from raw assessment scores to CLO/PLO attainment percentages, with every required edge case handled explicitly.
- **Real-time visibility and reporting**, so faculty see attainment update live and can export accreditation-ready reports on demand.

1.3 Definitions & Acronyms

Term	Meaning
OBE	Outcome-Based Education
CLO	Course Learning Outcome
PLO	Programme Learning Outcome
RBAC	Role-Based Access Control
JWT	JSON Web Token
ORM	Object-Relational Mapping
pgvector	PostgreSQL extension for native vector storage and similarity search
Cosine similarity	A measure of how close two embedding vectors point in the same direction, used to rank semantic closeness

2. SDLC Methodology & System Overview

2.1 SDLC Model

AdaptOBE was built using an **Agile, module-based incremental delivery model**, not a single big-bang implementation. The system specification itself is organized as a sequential roadmap of modules, and each module was executed as one iteration with a strict **definition of done**:

- Implement the module's models, services, and API endpoints.
- Write automated tests **alongside** the implementation, not retrofitted afterward, including every documented edge case.

- Run the full test suite and confirm all tests pass before moving to the next module.
- Commit with a module-referencing message, so the project history is itself a record of the iterations.

This produced eight completed iterations to date:

Module	Delivered
0	Local PostgreSQL 18 + pgvector, FastAPI/SQLAlchemy async skeleton
1	JWT auth, bcrypt hashing, RBAC middleware, Admin CRUD (users/departments/programmes)
2	Course/CLO/PLO management, AI semantic mapping engine
3	Assessments, scoring, the direct attainment engine
4	Faculty dashboard, real-time WebSockets, PDF/Excel export
—	Institutional data seed (UBIT department, 4 programmes, 10 PLOs)
—	Frontend UI expansion (Admin panel, course authoring, mapping UI, scoring UI)

Each iteration ended with a working, tested increment — the system has been demoable after every module, which is the core Agile property this methodology was chosen for.

2.2 Core System Objectives

- Replace manual, spreadsheet-driven CLO-to-PLO mapping with an AI-assisted, **human-in-the-loop** suggestion engine.
- Compute direct attainment correctly under every edge case an academic auditor would ask about (absent students, zero-denominator CLOs, threshold flagging).
- Give faculty **real-time** visibility into attainment without manual refresh.
- Enforce strict Role-Based Access Control so Admin, Faculty, and Student capabilities never overlap.
- Produce accreditation-ready PDF/Excel reports on demand.

2.3 Development Practices

- **Async-first backend:** every database interaction uses SQLAlchemy 2.0 AsyncSession; no blocking I/O in request handlers.
- **Layered separation:** routers contain no business logic — they call into a dedicated services layer, which is what makes the attainment math and AI matching independently unit-testable.
- **Schema-as-code:** every schema change is a reviewed Alembic migration, checked for drift (`alembic check`) before being considered complete.
- **Test-first edge cases:** the mathematical edge cases in Section 7 were written as pytest cases while the attainment engine was being built, per the project's own governing specification.

3. System Architecture

Diagram 1 — System Architecture Flowchart

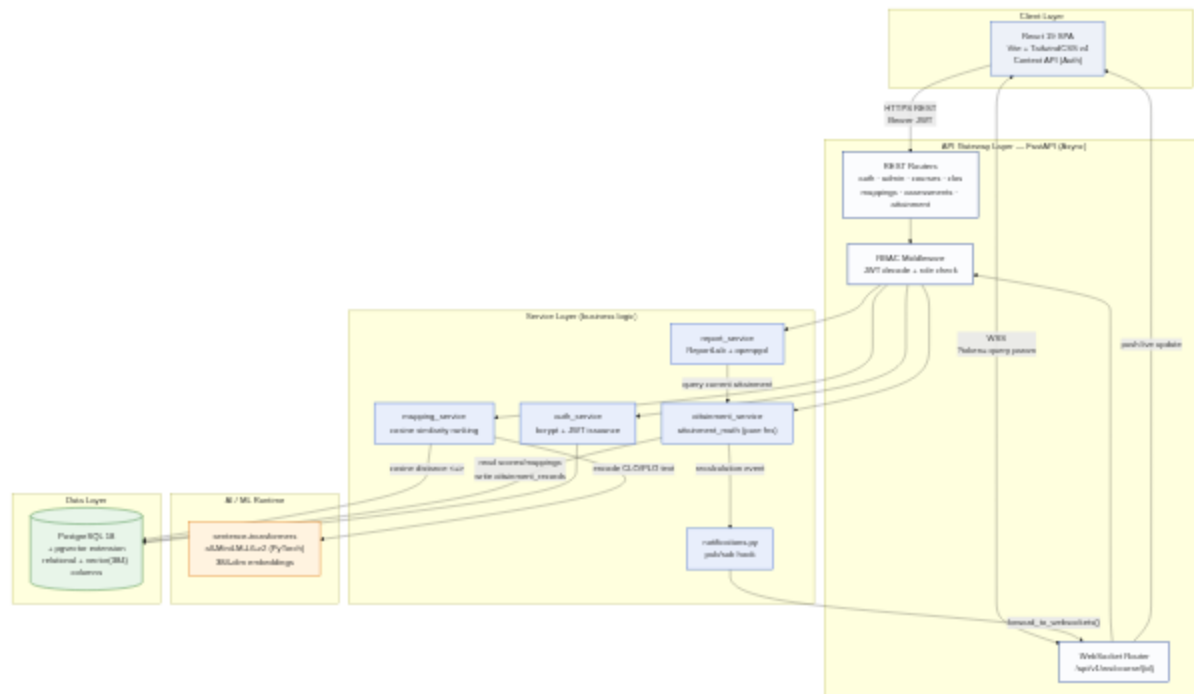


Figure 1. System Architecture Flowchart.

Design Decisions

- **Routers → Services → Data, strictly layered.** Routers only translate HTTP ↔ Pydantic schemas and catch typed service exceptions into HTTP status codes. All business logic — including the attainment math and the AI matching — lives in the service layer, which is what allows it to be unit-tested with zero HTTP or database mocking.
- **pgvector co-located with relational data,** not a separate vector database. CLO/PLO embeddings live as native vector(384) columns in the same PostgreSQL instance as the relational schema, avoiding a second system to keep in sync.
- **Native FastAPI WebSockets, not a message broker.** A single-process, in-memory ConnectionManager keyed by course_id is sufficient here and avoids the operational overhead of Redis pub/sub. The notifications.py module is deliberately decoupled from the WebSocket transport, so the attainment engine has no knowledge that WebSockets exist.
- **JWT access + refresh tokens,** validated per-request with no server-side session store, keeping the API layer stateless.
- **WebSocket authentication via query parameter.** Browsers cannot set custom headers on the WebSocket handshake, so the access token travels as ?token=, validated against the same RBAC and course-ownership rules as the REST endpoints.

4. Requirements & Use Cases

Diagram 2 — UML Use Case Diagram

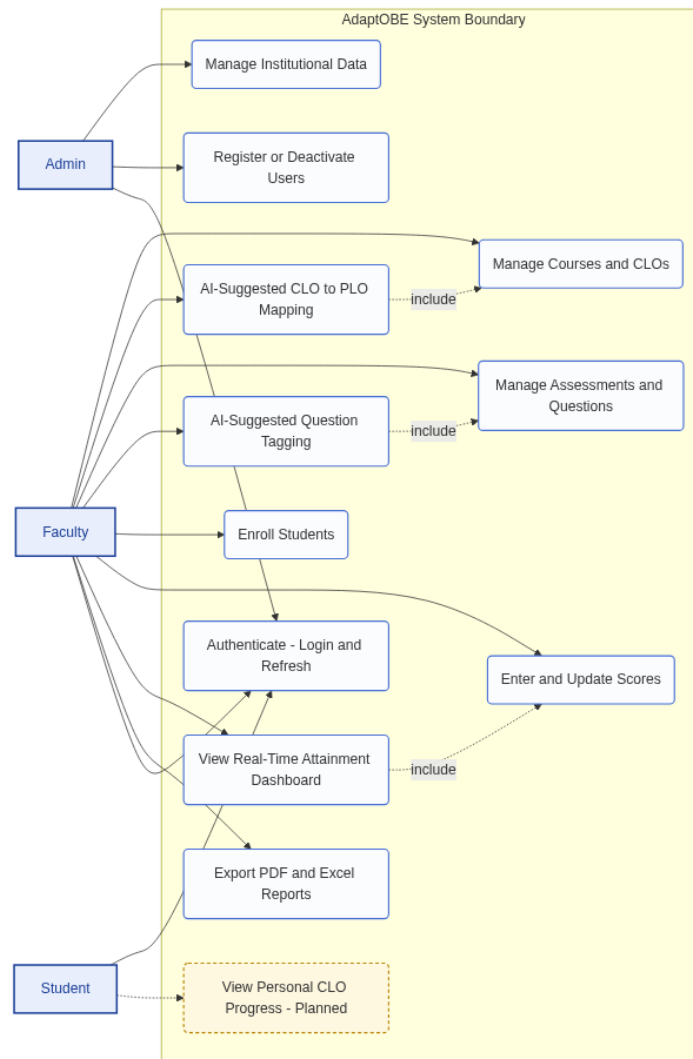


Figure 2. UML Use Case Diagram.

Design Decisions

- **Three actors map exactly onto the `user_role` enum** (admin, faculty, student) enforced by RBAC middleware — every use case here is backed by a `require_roles(...)` dependency on its endpoint.
- **UC12 is explicitly marked planned, not implemented.** Student accounts exist today (they can authenticate and be enrolled/scored by faculty), but there is no student-facing portal yet — that is Module 6 on the roadmap.
- **Include relationships** show real functional dependencies: AI-suggested mapping only exists inside CLO management; AI-suggested question tagging only exists inside assessment management; the live dashboard's headline metric is produced by score entry.

- **Admin and Faculty use cases don't overlap by accident** — courses, CLOs, and assessments are faculty-owned resources, while departments, programmes, PLOs, and user accounts are institution-level, admin-only resources.

Diagram 3 — Class / Entity Relationship Diagram



- **PLOs are stored per-programme, not shared.** `plos.program_id` is a foreign key, so with 4 fixed programmes and 10 standard outcomes there are 40 PLO rows, keeping the CLO→PLO suggestion query scoped to a single programme and letting one programme revise its wording later without disturbing the other three.
- **Cascade rules were fixed after a real bug.** Early on, `clos.course_id` and the mapping foreign keys had no ON DELETE behavior, so deleting a course with CLOs raised an unhandled 500. Courses now cascade-delete their CLOs and mappings; deleting a CLO uses SET NULL on `questions.clo_id` so exam records survive the tag being removed.
- **attainment_records is derived, not authoritative, data** — fully recomputed (delete-then-reinsert) on every score change rather than patched incrementally.

- **Every UNIQUE constraint does real work:** (clo_id, plo_id) prevents duplicate mappings, (course_id, student_id) prevents double enrollment, (question_id, student_id) prevents duplicate scores, and (student_id, course_id, clo_id) keeps exactly one attainment record per scope.
- **vector(384) matches the embedding model exactly** and is nullable so a CLO/PLO can exist before its embedding is generated rather than blocking the write.

6. AI Subsystem Design — Semantic CLO-to-PLO Mapping

Diagram 4 — Sequence Diagram: AI Semantic CLO to PLO Mapping

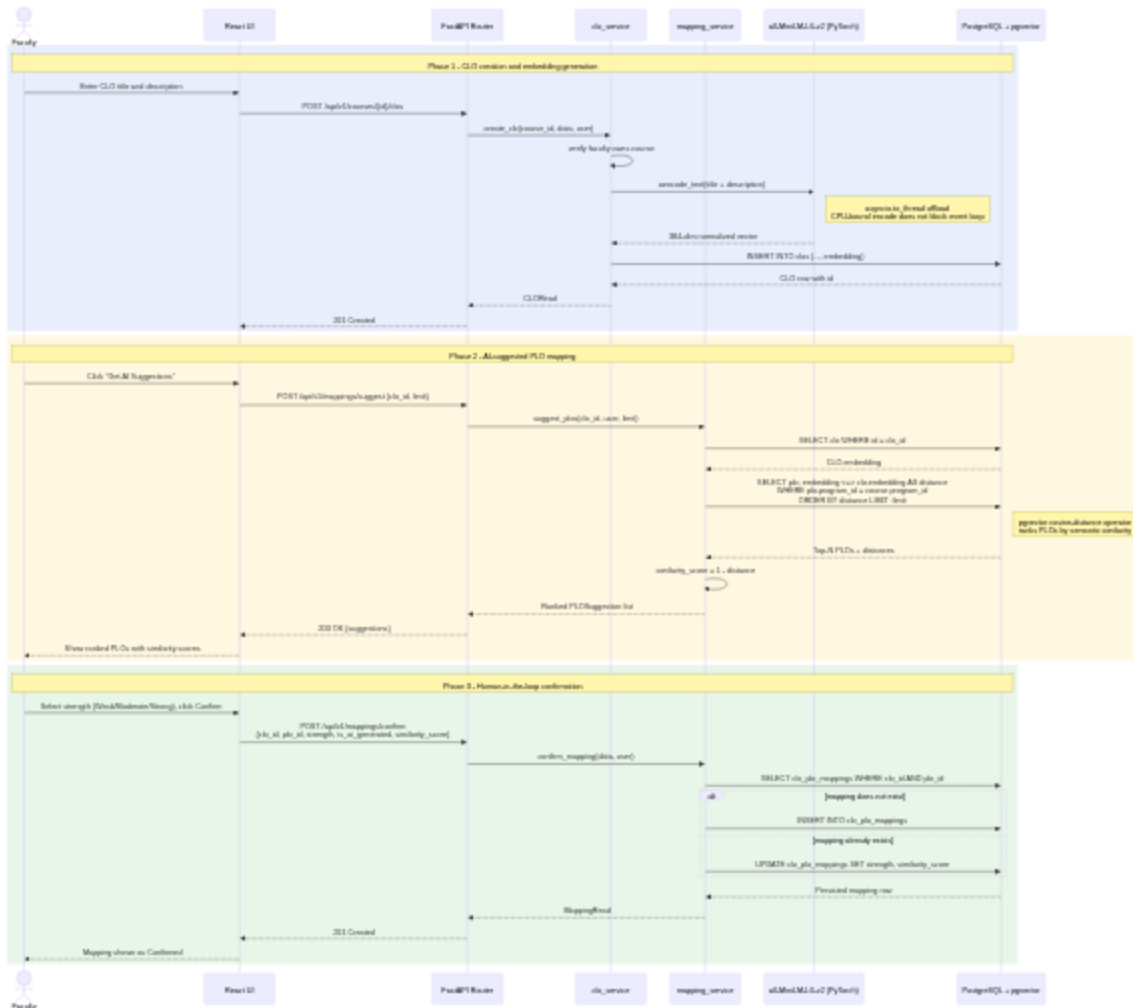


Figure 4. Sequence Diagram: AI Semantic CLO to PLO Mapping.

Design Decisions

- **Model choice: all-MiniLM-L6-v2.** Selected as the right accuracy/latency trade-off for short academic-outcome text at a fraction of the inference cost of a larger model — important because embedding happens synchronously inside a user-facing request.
- **Embeddings are computed once, at creation time,** not on every mapping request. Suggesting matches is a single indexed similarity query, not a re-encode of every PLO.
- **The CPU-bound encode is offloaded via `asyncio.to_thread`.** Running synchronous PyTorch inference directly inside an async route would block the entire event loop for every other concurrent request.
- **pgvector's `<=>` cosine-distance operator,** not `<->` (L2). Embeddings are L2-normalized at encode time, so `similarity_score = 1 minus cosine_distance` is a directly interpretable 0 to 1 score.

- **Human-in-the-loop by design.** /mappings/suggest is read-only; only an explicit faculty confirm action persists a mapping — semantic similarity is a strong suggestion signal, not a substitute for an academic's judgment.
- **Live-verified result:** for a CLO worded around agile teamwork, the engine correctly ranked the PLO “Individual and Team Work” first with 0.56 cosine similarity ahead of four other candidates, against the real seeded UBIT PLO set.

Diagram 5 — Activity Diagram: End-to-End Attainment and WebSocket Flow

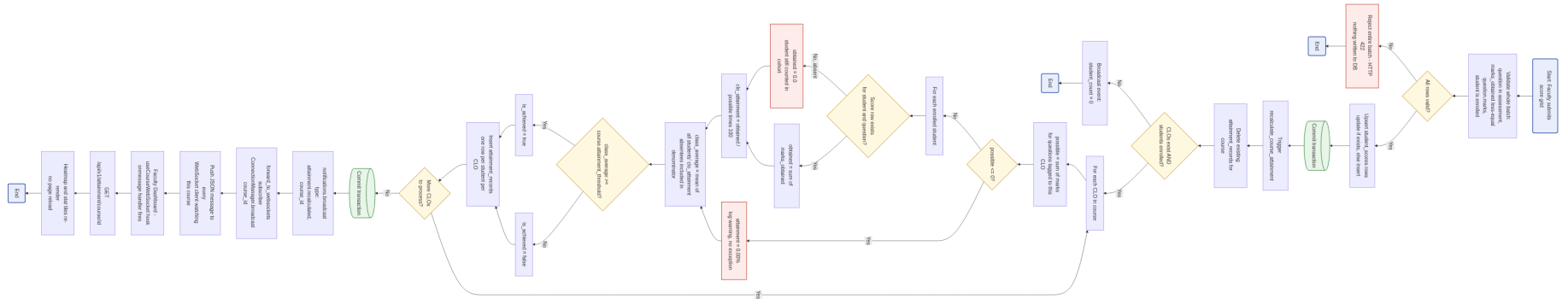


Figure 5. End-to-end activity flow, rotated for legibility on a landscape page — this is an inherently long, linear pipeline from score submission to a live dashboard update.

Design Decisions

- **The attainment math is a pure function**, taking numbers in and returning a percentage out — no database session, no HTTP context. This is what makes it possible to unit-test every edge case in isolation in milliseconds.
- **Validate the whole batch, then write — not row-by-row.** A single invalid row rejects the entire submitted grid, so faculty never end up with a half-entered assessment silently missing some students' scores.
- **Recalculation replaces, it does not patch.** attainment_records for a course are deleted and rebuilt from scratch on every recalculation, trading write volume for a much stronger correctness guarantee — no stale record can ever survive.
- **Absent students are a first-class case, not an omission.** A student with no score row is scored 0.0 and still counted in the class-average denominator.
- **The notification layer is decoupled from its transport.** attainment_service calls a generic notifications.broadcast(event) with no knowledge of WebSockets, so the real-time layer could be swapped later without touching the attainment engine.
- **Live-verified result:** entering a single new score during development moved a CLO's class average from 70.0% to 72.0% on an already-open dashboard in a separate browser tab, with zero manual refresh.

8. Non-Functional Requirements & Security

Concern	Implementation
Authentication	JWT access tokens (15 min) + refresh tokens (7 days) via python-jose. Frontend silently refreshes on a 401, coalesced across concurrent requests.
Password storage	bcrypt, salted per-user; never logged or returned in any API response.
Authorization (RBAC)	Every protected endpoint declares its allowed roles explicitly via a require_roles(...) dependency.
WebSocket authorization	Same RBAC and course-ownership checks as REST, applied at connection time via the ?token= query parameter.
CORS	Explicitly restricted to the known frontend origin, not wildcard-open.
Referential integrity	Cascade rules chosen per-relationship: cascade where child data is meaningless without its parent, SET NULL where it must outlive it.
Input validation	Every request body is a Pydantic v2 schema with field-level constraints validated before reaching business logic.

9. Testing Strategy

AdaptOBE has **165 automated backend tests**, all passing, organized in three tiers:

- **Pure unit tests** on the attainment math (24 tests) — no database, no HTTP, sub-second execution. Every edge case (zero-denominator, empty cohort, absentee handling, custom thresholds) has a dedicated test.

- **Service/engine integration tests** — exercise the attainment engine, the AI mapping engine, and the WebSocket ConnectionManager against a real, transactionally isolated and rolled back, PostgreSQL database.
- **API/RBAC tests** — drive every endpoint through its full HTTP surface, asserting both the happy path and that each role boundary actually rejects the roles it should.

Two real bugs were caught by this process rather than by manual testing, both fixed and covered by a regression test immediately: a missing ON DELETE CASCADE that caused a 500 when deleting a course with CLOs, and two RBAC routers that were accidentally admin-only end-to-end when faculty needed read access to function at all.

10. Implementation Status & Roadmap

Completed and demo-verified: Authentication & RBAC · Institutional data management · Course & CLO management · AI-powered semantic CLO→PLO mapping · Assessments, questions & AI-assisted tagging · The direct attainment engine · The real-time faculty dashboard · PDF/Excel export · Full Admin and Faculty user interfaces.

Institutional data seeded: the UBIT department, its four degree programmes (BS Software Engineering, BS Computer Science, BS Artificial Intelligence, BS Data Science), and the ten standard Programme Learning Outcomes under each — 40 PLO rows, each with a real generated embedding.

Planned (not yet built):

- **Module 5** — ML-based student risk prediction (XGBoost) with SHAP explainability.
- **Module 6** — a dedicated student self-service portal (accounts exist today; the portal UI does not).
- **Module 6** — an adaptive quiz generator driven by per-student CLO performance.
- **Module 7** — production containerization (Docker) and deployment.

11. Appendix — Technology Stack

Layer	Technology
Backend	Python 3.11+, FastAPI, fully async architecture
Database	PostgreSQL 18 + pgvector extension
ORM / Migrations	SQLAlchemy 2.0 (AsyncSession) + Alembic
Validation	Pydantic v2
Authentication	JWT (python-jose) + bcrypt
AI / ML runtime	PyTorch + sentence-transformers (all-MiniLM-L6-v2)
Frontend	React.js (JavaScript, no TypeScript) + TailwindCSS v4, React Context for state
Real-time	Native FastAPI WebSockets
Reporting	ReportLab (PDF), openpyxl (Excel)

Layer	Technology
Testing	pytest, pytest-asyncio, httpx